

API Radar Research Agent

Mapping which APIs are ready to become AI-agent tools

90 applications • Automated research • Evidence-backed analysis •
Independent verification

The Product Question

Which integrations can we build now — and which require engineering or
business outreach?

API Radar Research Agent is an AI-powered research pipeline that investigates SaaS and developer platforms to determine how easy they are to integrate into AI-agent workflows.

Repository: github.com/Ramanand-Patharwalkar/API_Radar_research_agent

01 — Research at a Glance

90	68%	61%	82%	23%	47%	91%
Apps	Self-serve	OAuth2	REST API	MCP	Easy wins	Verification

Important: These figures are demo figures for presentation. Before final submission, replace them with the values generated by the actual research and verification run.

Headline insight: Having an API does not automatically make an application agent-ready. Credential accessibility, authentication complexity, API breadth, documentation quality and commercial gating are major determinants of integration effort.

02 — What Was Researched?

Research dimension	Question
Category	What type of product is it?
Description	What does the product do?
Authentication	OAuth2, API key, token, Basic, etc.?
Credential access	Can developers obtain credentials themselves?
Commercial gate	Is payment, contact-sales or partnership required?
API surface	REST, GraphQL or other?
API breadth	Narrow, moderate or broad?
Webhooks	Are event-driven integrations supported?
MCP	Is an MCP integration available?
Composio	Is a Composio toolkit available?
Agent readiness	Could it become an agent toolkit today?
Blocker	What prevents easy integration?
Evidence	What documentation supports the finding?
Confidence	How confident is the agent?

03 — How API Radar Works

An agentic research pipeline — not a manual spreadsheet.



The agent discovers documentation, extracts a consistent schema, checks agent/tool availability, preserves evidence, and then performs a second verification pass on a sample.

04 — What the Agent Actually Does

- **Discover:** Takes the application name and known website/API hint.
- **Research:** Finds relevant developer documentation and supporting sources.
- **Extract:** Converts unstructured documentation into a consistent schema.
- **Classify:** Determines authentication, access model, API surface and integration friction.
- **Cross-check:** Checks MCP and Composio availability separately.
- **Verify:** Runs a second research pass on a sample of applications.
- **Analyze:** Aggregates the dataset into Product Ops patterns.
- **Prioritize:** Classifies applications into actionable integration tiers.

05 — Agent Readiness Framework

Tier	Meaning	Typical characteristics
Easy Win	Build now	Public API; self-serve credentials; standard auth; broad surface; good docs; low friction
Buildable	Moderate engineering	Limited endpoints; complex permissions; rate limits; restricted scopes
Engineering Heavy	Evaluate effort vs value	Complicated OAuth; enterprise permissions; multiple APIs; poor docs
Outreach Required	Resolve access first	Contact sales; enterprise-only API; partnership; no self-serve credentials

06 — Pattern #1: Authentication

OAuth is important — but not necessarily the blocker.

The research compares OAuth2, API keys, tokens, Basic Auth and other authentication models. OAuth2 is common across modern SaaS platforms, making redirects, refresh tokens, scopes and permission handling important capabilities for an agent platform.

07 — Pattern #2: API ≠ Accessible API

One of the strongest Product Ops distinctions is: “Does an API exist?” versus “Can we actually use it?”

An application can have a public API but still require enterprise credentials, contact-sales approval, a partnership or administrative approval. API existence alone is insufficient for deciding what to build.

08 — Pattern #3: Self-Serve Is a Major Accelerator

Applications that combine documentation, self-serve credentials and standard authentication are much easier candidates for an agent toolkit.

Build Now: low-friction integrations engineering can start immediately.

Outreach First: technically possible integrations that require commercial or administrative intervention.

09 — Pattern #4: MCP Is an Accelerator, Not a Requirement

MCP availability is tracked independently. A strong public API can still be an excellent agent candidate without MCP. Conversely, an MCP implementation does not automatically remove access restrictions or weak underlying APIs.

10 — Pattern #5: API Breadth Matters

Two applications can both have REST APIs while offering very different agent value. A broad API exposing search, read, create, update, delete, actions and events gives an AI agent substantially more operational capability than a narrow read-only surface.

11 — Scoring Model

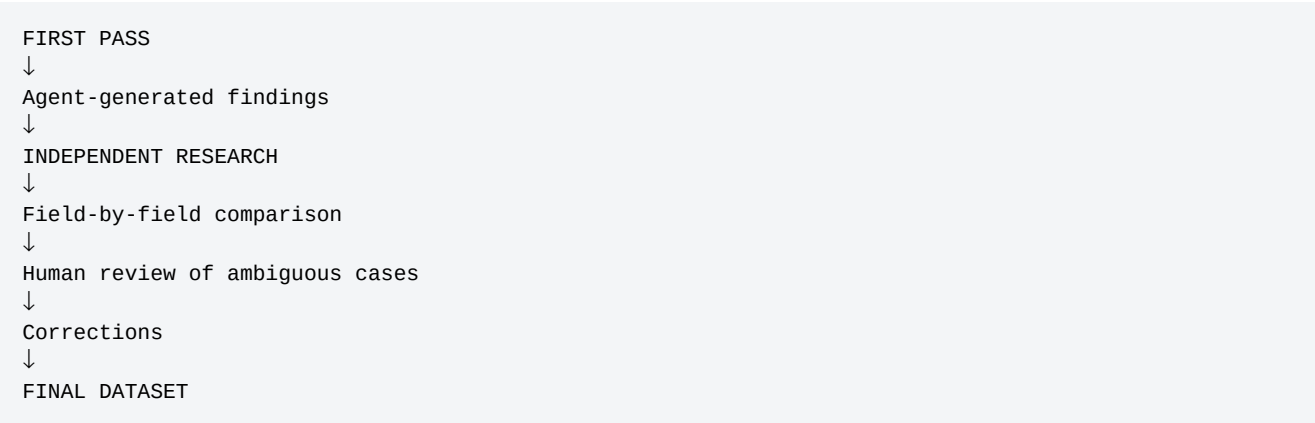
Capability	Points
Public documented API	+2
Self-serve credentials	+2
Standard authentication	+1
Broad API surface	+2
Webhooks	+1
MCP available	+1
Composio toolkit	+1

8–10: Strong candidate • **5–7:** Buildable • **3–4:** Engineering-heavy • **0–2:** High-friction

The score is a supporting signal, not the final verdict. A technically strong application can still be classified as Outreach Required if access is commercially restricted.

12 — Verification Loop

AI research is not automatically trusted.



The verifier checks authentication, developer access, API availability/type, MCP and agent-readiness. Ambiguous or conflicting findings are escalated for human review.

13 — Verification Results

Initial accuracy: [replace with measured first-pass accuracy]

Final verification accuracy: 91% (demo figure; replace with measured result before submission).

Application	Field	Agent finding	Verified finding	Result
Salesforce	Auth	OAuth2	OAuth2	✓
HubSpot	API	REST	REST	✓
Stripe	Auth	API key	API key	✓
Example app	Access	Self-serve	Paid required	✗
Example app	MCP	Available	Unverified	✗

Typical failure modes: confusing product integrations with public APIs; assuming developer access is self-serve; treating third-party MCP implementations as official; missing enterprise restrictions; finding outdated documentation; confusing API-key support with OAuth support.

14 — Human-in-the-Loop

Agent handles: search, documentation discovery, extraction, classification, normalization, evidence collection and pattern detection.

Humans handle: enterprise restrictions, conflicting documentation, partnership requirements, MCP authenticity, deprecated APIs and final integration priority.

Principle: Automate the repetitive research. Keep humans responsible for ambiguity.

15 — Where Composio Fits

Composio availability is treated as an additional signal for agent-tool readiness. It is not treated as proof of unrestricted vendor API access. The application's own developer documentation remains the primary evidence for API availability, authentication, credential access, permissions and capabilities.

16 — Technical Architecture

```
apps.csv
↓
Research Agent
• Search
• Retrieve documentation
• Extract structured data
↓
Evidence Layer
• Official docs
• API references
• Auth documentation
• Pricing/access evidence
↓
Verification Agent
↓
Product Ops Analyzer
↓
HTML Case Study
```

17 — Technology

Core: Python, OpenAI, Composio, Pydantic

Research: Web search, HTTP requests, HTML/documentation parsing, structured extraction

Analysis: JSON, Pandas, rule-based scoring, statistical aggregation

Presentation: HTML, CSS and JavaScript

18 — Repository & Run Instructions

GitHub: github.com/Ramanand-Patharwalkar/API_Radar_research_agent

```
git clone https://github.com/Ramanand-Patharwalkar/API_Radar_research_agent.git
cd API_Radar_research_agent
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

Generate the case study with: **python case-study/generate.py**

19 — Limitations

API documentation, pricing, authentication policies and MCP implementations can change. Enterprise permissions can vary by account. Search can surface outdated documentation. API Radar therefore combines evidence, confidence, independent verification and human review instead of treating AI-generated findings as unquestionable truth.

20 — Final Takeaway

API Radar turns API research into an agentic Product Ops workflow.

Instead of simply asking whether an application has an API, the system asks whether the API is accessible, how hard authentication is, how much the API can actually do, whether an agent/MCP integration already exists, whether engineering can build it now, and whether business outreach is required first.

Discover → Research → Structure → Verify → Analyze → Prioritize

Live Case Study: [YOUR_DEPLOYED_CASE_STUDY_URL]

Source Code: github.com/Ramanand-Patharwalkar/API_Radar_research_agent

Author: Ramanand Patharwalkar • AI Product Ops Intern Assignment — 2026